Lecture 35

STAT 109: Introductory Biostatistics - Simple linear regression (part 1)

Lecture 35

From scatterplot to simple linear regression models

In Lecture 26, for two numerical variables we read a scatterplot using:

- form (linear or curved or neither),
- direction (positive or negative or none),
- strength (tight or loose around a trend).

When the form looks roughly linear, we can summarize linear relationship with:

- Pearson correlation `r`, and
- a least-squares regression line.

Learning objectives

By the end of this lecture, you should be able to:

- read a scatterplot using form, direction, and strength;
- compute and interpret Pearson correlation for two numerical variables;
- fit and interpret a simple linear regression line (`lm`);
- interpret slope and intercept in context;
- make predictions for new `x` values using the fitted equation;
- compare training vs test performance using residual summaries, MSE, and `R^2`.

Pearson correlation and least-squares line

For a response `y` and explanatory variable `x`:

- Pearson Correlation (`cor`) is unitless and ranges from `-1` to `1` and is a one number summary of the direction and strength of a linear relationship between two numerical variables (assumes that the form is linear)
- The fitted line is:

$$\hat{y} = b_0 + b_1 x$$

where `b1` is slope and `b0` is intercept.

In R, fit with `lm(y ~ x, data = ...)`.

Practice activity (visualizing best-fit line choices by error criterion, including MSE/MAE):
https://www.rossmanchance.com/applets/2021/regshuffle/regshuffle.htm

Sample correlation formula and interpretation

For sample data `(x_i, y_i), i = 1, ..., n`, the sample correlation is

$$r = \frac{\sum_{i=1}^{n}(x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^{n}(x_i - \bar{x})^2}\sqrt{\sum_{i=1}^{n}(y_i - \bar{y})^2}}$$

How to read this:

- **Numerator:** adds products `(x_i - xbar)(y_i - ybar)`.
  - If `x` and `y` are often both above their means (or both below), products tend to be positive, so `r` tends to be positive (positive direction).
  - If one is often above mean while the other is below mean, products tend to be negative, so `r` tends to be negative (negative direction).
- **Denominator:** rescales by overall spread in `x` and `y`, which keeps `r` between `-1` and `1`.
- `r` near `0` means no clear **linear** direction.

Quick interpretation reminder:

- If `|r|` is closer to `1` (for example `-0.9` or `0.9`), the linear relationship is stronger.
- If `r` is close to `0`, the linear relationship is weak to none.
- The sign gives direction:
  - $r > 0$: positive direction (as `x` increases, `y` tends to increase).
  - $r < 0$: negative direction (as `x` increases, `y` tends to decrease).

Practice activity: Rossman Chance "Guess the Correlation"
https://www.rossmanchance.com/applets/guesscorrelation/GuessCorrelation.html

Condition reminder for using correlation

Pearson correlation is a linear-association summary. Use it when the scatterplot form is roughly linear (not strongly curved and not dominated by outliers).

Example 1 - Study hours and exam score

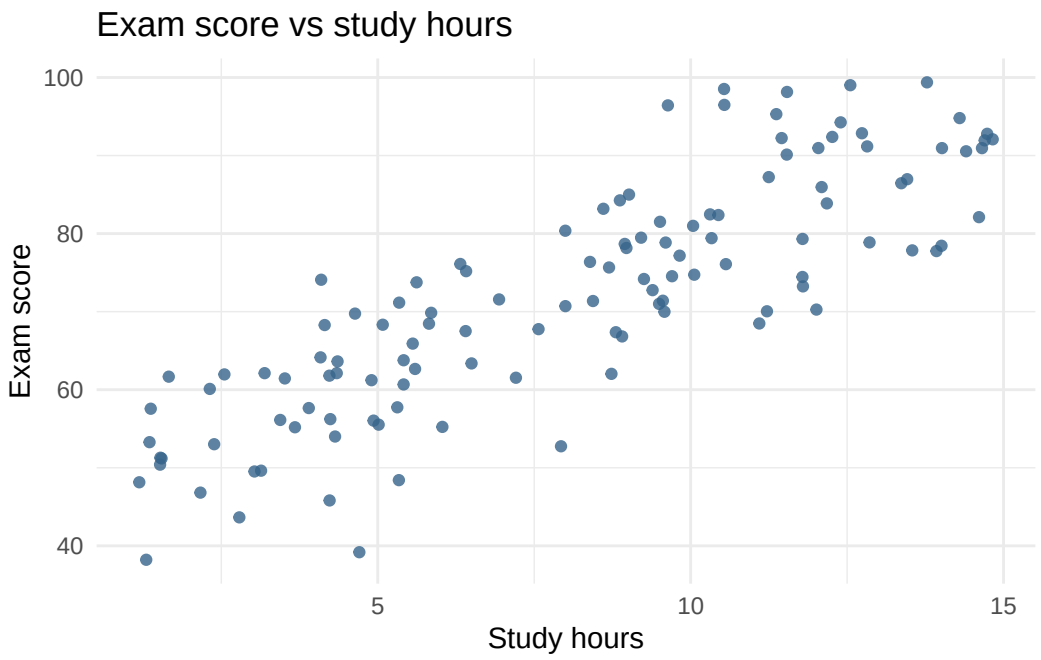
```
if (!requireNamespace("tidyverse", quietly = TRUE)) {
  install.packages("tidyverse", repos = "https://cloud.r-project.org")
}

library(tidyverse)

set.seed(3401)
n <- 120
exam_dat <- tibble(
  study_hours = runif(n, 1, 15),
  exam_score = 45 + 3.4 * study_hours + rnorm(n, 0, 8)
)
```

1) Scatterplot: form, direction, strength

```
ggplot(exam_dat, aes(x = study_hours, y = exam_score)) +
  geom_point(alpha = 0.8, color = "steelblue4") +
  labs(
    title = "Exam score vs study hours",
    x = "Study hours",
    y = "Exam score"
  ) +
  theme_minimal()
```



Interpretation: the relationship is roughly linear, positive, and moderate-to-strong.

2) Pearson correlation

```
cor(exam_dat$study_hours, exam_dat$exam_score)
```

```
[1] 0.8441325
```

3) Training and test split (80/20)

```
set.seed(3402)
idx_train <- sample(seq_len(nrow(exam_dat)), size = 0.8 * nrow(exam_dat))
exam_train <- exam_dat[idx_train, ]
exam_test  <- exam_dat[-idx_train, ]

nrow(exam_train)
```

```
[1] 96
```

```
nrow(exam_test)
```

```
[1] 24
```

4) Fit least-squares regression on training data

```
fit_exam <- lm(exam_score ~ study_hours, data = exam_train)
summary(fit_exam)
```

```

Call:
lm(formula = exam_score ~ study_hours, data = exam_train)

Residuals:
```

```
     Min       1Q   Median       3Q      Max
-19.1928  -4.7092  -0.2337   5.4140  16.5633

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept)  47.6895     1.7424   27.37   <2e-16 ***
study_hours   3.0594     0.1906   16.05   <2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 7.519 on 94 degrees of freedom
Multiple R-squared:  0.7326,    Adjusted R-squared:  0.7298
F-statistic: 257.5 on 1 and 94 DF,  p-value: < 2.2e-16
```

```r
coef(fit_exam)
```

```
(Intercept) study_hours
  47.689527    3.059362
```

Interpretation:

- slope `b1`: predicted increase in exam score is 3.06 points for each additional hour of study.
- intercept `b0`: predicted score is 47.7 when study hours = 0

```r
new_hours <- tibble(study_hours = 10)
predict(fit_exam, newdata = new_hours)
```

```
       1
78.28315
```

If a student studies 10 hours then our linear model predicts they will get an exam score of 78.3. By hand, $\hat{score} = 47.7 + 3.06(10)$.

Residuals and model performance: training vs test

Residual is `observed y - predicted y`.

```r
exam_train <- exam_train |>
  mutate(
    pred = predict(fit_exam, newdata = exam_train),
    resid = exam_score - pred
  )

exam_test <- exam_test |>
  mutate(
    pred = predict(fit_exam, newdata = exam_test),
    resid = exam_score - pred
  )
```

Residual descriptives for both sets

```r
resid_desc <- bind_rows(
  exam_train |>
```

```
    summarise(
      set = "train",
      n = n(),
      mean_resid = mean(resid),
      sd_resid = sd(resid),
      min_resid = min(resid),
      q1_resid = quantile(resid, 0.25),
      median_resid = median(resid),
      q3_resid = quantile(resid, 0.75),
      max_resid = max(resid)
    ),
  exam_test |>
    summarise(
      set = "test",
      n = n(),
      mean_resid = mean(resid),
      sd_resid = sd(resid),
      min_resid = min(resid),
      q1_resid = quantile(resid, 0.25),
      median_resid = median(resid),
      q3_resid = quantile(resid, 0.75),
      max_resid = max(resid)
    )
)
resid_desc
```

```
# A tibble: 2 x 9
  set       n mean_resid sd_resid min_resid q1_resid median_resid q3_resid
  <chr> <int>      <dbl>    <dbl>     <dbl>    <dbl>        <dbl>    <dbl>
1 train    96   1.78e-15     7.48     -19.2    -4.71       -0.234     5.41
2 test     24  -1.82e+ 0     9.61     -22.9    -6.87       -1.57      2.80
# i 1 more variable: max_resid <dbl>
```

MSE and R-squared on training and test

```
mse <- function(y, yhat) mean((y - yhat)^2)

mse_train <- mse(exam_train$exam_score, exam_train$pred)
mse_test  <- mse(exam_test$exam_score, exam_test$pred)

ss_tot_train <- sum((exam_train$exam_score - mean(exam_train$exam_score))^2)
ss_tot_test  <- sum((exam_test$exam_score - mean(exam_test$exam_score))^2)
ss_res_train <- sum((exam_train$exam_score - exam_train$pred)^2)
ss_res_test  <- sum((exam_test$exam_score - exam_test$pred)^2)

r2_train <- 1 - ss_res_train / ss_tot_train
r2_test  <- 1 - ss_res_test / ss_tot_test

tibble(
  set = c("train", "test"),
  MSE = c(mse_train, mse_test),
```

```
  R2 = c(r2_train, r2_test)
)
```

```
# A tibble: 2 x 3
  set     MSE    R2
  <chr> <dbl> <dbl>
1 train  55.4 0.733
2 test   91.8 0.623
```

Interpretation notes:

- Training MSE is optimistically biased (it is measured on the same data used to fit the model).
- Test MSE is a less biased estimate of prediction error on new data.
- `R^2` is a rescaled error summary: roughly, the proportion of variability in `y` explained by linear prediction from `x`.
- Test `R^2` should be reported for expected out-of-sample performance.

Example 2 - Fertilizer dose and plant height

```
set.seed(3403)
plant_dat <- tibble(
  dose = runif(90, 0, 12),
  height_cm = 18 + 2.1 * dose + rnorm(90, 0, 5)
)

cor(plant_dat$dose, plant_dat$height_cm)
```

```
[1] 0.8321912
```

```
idx <- sample(seq_len(nrow(plant_dat)), size = 0.8 * nrow(plant_dat))
plant_train <- plant_dat[idx, ]
plant_test  <- plant_dat[-idx, ]

fit_plant <- lm(height_cm ~ dose, data = plant_train)
coef(fit_plant)
```

```
(Intercept)        dose
  17.737388    2.359837
```

```
plant_test <- plant_test |>
  mutate(pred = predict(fit_plant, newdata = plant_test))

tibble(
  test_MSE = mse(plant_test$height_cm, plant_test$pred),
  test_R2 = 1 - sum((plant_test$height_cm - plant_test$pred)^2) /
    sum((plant_test$height_cm - mean(plant_test$height_cm))^2)
)
```

```
# A tibble: 1 x 2
  test_MSE test_R2
     <dbl>   <dbl>
1     25.7   0.651
```

Example 3 - Exercise minutes and resting heart rate

```r
set.seed(3404)
heart_dat <- tibble(
  exercise_min = runif(100, 0, 60),
  resting_hr = 78 - 0.28 * exercise_min + rnorm(100, 0, 4.2)
)

cor(heart_dat$exercise_min, heart_dat$resting_hr)
```

```
[1] -0.780774
```

```r
idx <- sample(seq_len(nrow(heart_dat)), size = 0.8 * nrow(heart_dat))
heart_train <- heart_dat[idx, ]
heart_test  <- heart_dat[-idx, ]

fit_heart <- lm(resting_hr ~ exercise_min, data = heart_train)
coef(fit_heart)
```

```
 (Intercept) exercise_min
   76.526712    -0.247786
```

```r
heart_test <- heart_test |>
  mutate(pred = predict(fit_heart, newdata = heart_test))

tibble(
  test_MSE = mse(heart_test$resting_hr, heart_test$pred),
  test_R2 = 1 - sum((heart_test$resting_hr - heart_test$pred)^2) /
    sum((heart_test$resting_hr - mean(heart_test$resting_hr))^2)
)
```

```
# A tibble: 1 x 2
  test_MSE test_R2
     <dbl>   <dbl>
1     18.0   0.672
```